

Lecture 12 — The Gradebook Application

Hitting the wall in a design, and refactoring your way out

Amir Farbin

Where we left off

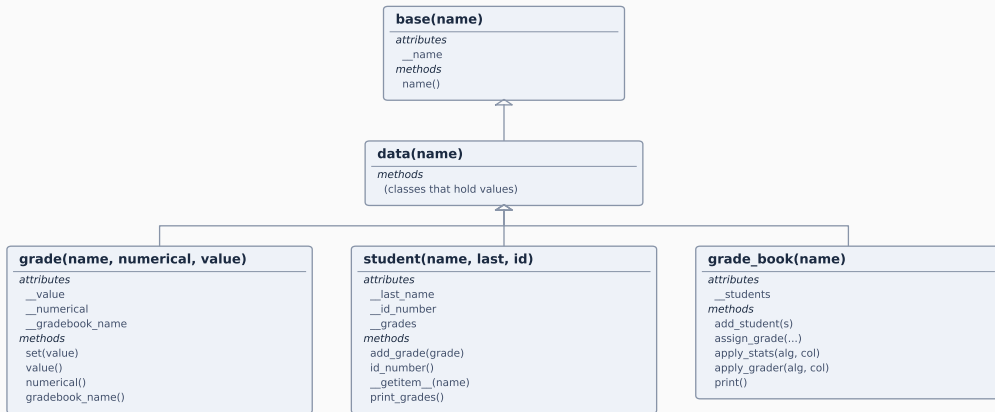
The story so far

- Lecture 11: a messy file → clean data → grading schemes → a first design
- Two kinds of class, kept apart on purpose:
 - **data**: grade, student, grade_book — they hold values
 - **algorithms**: the calculators — they hold only settings, and do work
- Today: the design is used in anger, and it **pushes back**

→ cells 0–2

Last lecture's classes, rebuilt

Data classes: they hold values



Every class carries a name, so an instance can be found and reported on.

Same code, quickly

- `base / data / alg`, then `grade`, `student`
- `grade_calculator`, `statistics_calculator`, `summary_calculator`
- `grade_book` with `apply_stats`, `apply_grader`, `apply_summary`
- Nothing new yet — we need it all on the table before we can criticise it

→ cells 4, 6, 8, 10, 13, 15, 17

Same code, quickly — cell 4

```
# Create some virtual classes
class base:
    __name=""

    def __init__(self,name):
        self.__name=name

    def name(self):
        return self.__name

class data(base):
    def __init__(self,name):
        base.__init__(self,name)

... (3 more lines)
```

Same code, quickly — cell 6

```
class grade(data):
    __letter_grades=["F-", "F", "F+", "D-", "D", "D+", "C-", "C", "C+", "B-", "B", "B+", "...

    def __init__(self, name, numerical=True, value=None):
        self.__value=value
        self.__numerical=numerical
        self.__gradebook_name=str()

        if value:
            if isinstance(value, (int, float)):
                self.__numerical=True
            elif isinstance(value, str):
                self.__numerical=False
            self.set(value)

... (26 more lines)
```

Same code, quickly — cell 8

```
class student(data):
    def __init__(self, first_name, last_name, id_number):
        self.__grades=dict()
        self.__id_number=id_number
        data.__init__(self,first_name+" "+last_name+" Student Data")

    def add_grade(self,a_grade,overwrite=False):
        if overwrite or not a_grade.gradebook_name() in self.__grades:
            self.__grades[a_grade.gradebook_name()]=a_grade
        else:
            print (self.name()+" Error Adding Grade "+a_grade.name()+". Grade ...
                raise Exception

    def id_number(self):
... (14 more lines)
```

Same code, quickly — cell 10

```
class grade_calculator(alg):
    def __init__(self,name,stats):
        self.__stats=stats
        alg.__init__(self,name)

    def apply(self,a_grade):
        raise NotImplementedError

class uncurved_letter_grade_percent(grade_calculator):
    __grades_definition=[ (.97,"A+"),
                           (.93,"A"),
                           (.9,"A-"),
                           (.87,"B+"),
                           ... (38 more lines)
```

Same code, quickly — cell 13

```
import numpy as np
import math

class statistics_calculator(alg):
    def __init__(self, name):
        alg.__init__(self, name)

    def apply(self, grades):
        raise NotImplementedError

class mean_std_calculator(statistics_calculator):
    def __init__(self):
        statistics_calculator.__init__(self, "Mean and Standard Deviation Calcu...

... (2 more lines)
```

Same code, quickly — cell 15

```
class summary_calculator(alg):
    def __init__(self,name):
        alg.__init__(self,name)

    def apply(self,a_student):
        raise NotImplementedError

class grade_summer(summary_calculator):
    def __init__(self,prefix,n):
        self.__prefix=prefix
        self.__n=n
        statistics_calculator.__init__(self,"Sum Grades")

    def apply(self,a_student):
... (7 more lines)
```

Same code, quickly — cell 17

```
class grade_book(data):  
    # New member class to hold arbitrary data associated with the class  
    def __init__(self,name):  
        data.__init__(self,name+" Course Grade Book")  
        self.__students=dict()  
  
    # New method to add data  
    def add_student(self,a_student):  
        self.__students[a_student.id_number()]=a_student  
  
    # New method to allow iterating over students  
    def get_students(self):  
        return self.__students  
  
... (50 more lines)
```

Loading it up

- `build_grade_book` — rows become students, columns become grades
- It has to redo Lecture 11's cleaning: the data comes in as text again
 - a lesson in itself: the fix lived in a notebook, not in the code
- Run a grader, run a summer, print the students

→ cells 24, 25, 27, 29, 30

```
a_grade_book=build_grade_book(df_0)
```

```
a_grade_book.print_students()
```

```
0 Student 0 Student Data
```

```
Lab 1: 0.0
```

```
Lab 2: 0.0
```

```
Lab 3: 0.0
```

```
Lab 4: 0.0
```

```
Exam 1: 0.0
```

```
Lab 5: 0.0
```

```
Unnamed: 6: 0.0
```

```
Lab 7: 0.0
```

```
... (183 more lines)
```

```
a_grade_book.apply_grader(curved_letter_grade("Lab 2 Letter",Lab_2_stats[0],La...
```

```
a_grade_book.apply_summary(grade_summer("Lab ",5))
```

```
a_grade_book.print_students()
```

```
0 Student 0 Student Data
```

```
Lab 1: 0.0
```

```
Lab 2: 0.0
```

```
Lab 3: 0.0
```

```
Lab 4: 0.0
```

```
Exam 1: 0.0
```

```
Lab 5: 0.0
```

```
Unnamed: 6: 0.0
```

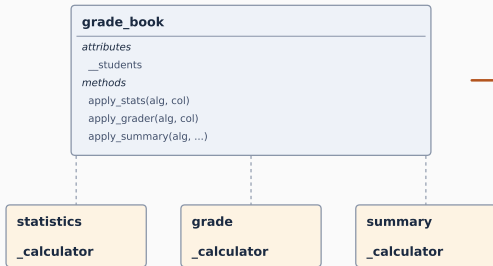
```
Lab 7: 0.0
```

```
... (215 more lines)
```

The wall

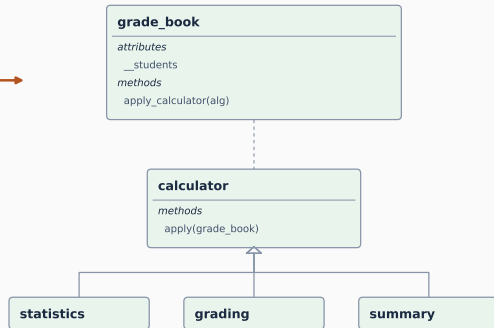
Three kinds, three methods

Lecture 11: three kinds, three methods



the caller has to know which kind it holds

Lecture 12: one interface



the grade book no longer cares

What's actually wrong

- `apply_stats`, `apply_grader`, `apply_summary` — nearly the same code three times
- They differ **only because we decided they should**
- Want to curve *and then* normalize? There's no way to compose them
- Every new kind of calculation means a new method on `grade_book`
- The caller has to know which kind it's holding — that kills polymorphism

The rule being broken

- The class should decide what it does; the caller shouldn't have to
- One name, many implementations — that's the whole point of a base class
- We had it in Lecture 9 with `Shape.paint`, and lost it here
- Fix: make every algorithm look the same from outside

The refactor

A blackboard

- Let the grade book hold **arbitrary results**, not just students' grades
- `__getitem__` / `__setitem__` — the book behaves like a dictionary
- One algorithm writes Lab 2 stats; the next one reads it
- This is how a particle-physics experiment's analysis software is organised, and most data pipelines besides

→ cell 39

A blackboard — cell 39

```
class grade_book(data):  
    # New member class to hold arbitrary data associated with the class  
    def __init__(self, name):  
        data.__init__(self, name+" Course Grade Book")  
        self.__students=dict()  
        self.__data=dict()  
  
    # New method to access data  
    def __getitem__(self, key):  
        return self.__data[key]  
  
    # New method to add data  
    def __setitem__(self, key, value):  
        self.__data[key] = value  
... (68 more lines)
```

One interface

- `calculator.apply(grade_book, **kwargs)`, and raise `NotImplementedError` in the base
- `grade_book.apply_calculator(calc)` — one method, forever
- The book no longer knows or cares what kind of calculation it was handed
- Adding a calculation adds a class. Nothing else changes.

→ cells 37, 43, 46, 47

```
class calculator(alg):  
    def __init__(self,name):  
        alg.__init__(self,name)  
  
    def apply(self,a_grade_book):  
        raise NotImplementedError
```

One interface — cell 43

```
class uncurved_letter_grade_percent(calculator):  
    __grades_definition=[ (.97,"A+"),  
                           (.93,"A"),  
                           (.9,"A-"),  
                           (.87,"B+"),  
                           (.83,"B"),  
                           (.8,"B-"),  
                           (.77,"C+"),  
                           (.73,"C"),  
                           (.7,"C-"),  
                           (.67,"C+"),  
                           (.63,"C"),  
                           (.6,"C-"),  
                           (.57,"F+"),  
  
    ... (33 more lines)
```

One interface — cell 46

```
a_grade_book.apply_calculator(uncurved_letter_grade_percent("Lab 2",max_grade=...
```

```
for k,a_student in a_grade_book.get_students().items():  
    print (a_student.id_number(),a_student["Lab 2"],a_student["Lab 2 Letter"])
```

```
0 Lab 2: 0.0 Lab 2 Letter: F-  
1 Lab 2: 96.0 Lab 2 Letter: A  
2 Lab 2: 100.0 Lab 2 Letter: A+  
3 Lab 2: 100.0 Lab 2 Letter: A+  
4 Lab 2: 81.0 Lab 2 Letter: B-  
5 Lab 2: 100.0 Lab 2 Letter: A+  
6 Lab 2: 98.0 Lab 2 Letter: A+  
7 Lab 2: 98.0 Lab 2 Letter: A+  
8 Lab 2: 96.0 Lab 2 Letter: A  
... (7 more lines)
```

```
a_grade_book.print_students()
```

```
0 Student 0 Student Data
```

```
Lab 1: 0.0
```

```
Lab 2: 0.0
```

```
Lab 3: 0.0
```

```
Lab 4: 0.0
```

```
Exam 1: 0.0
```

```
Lab 5: 0.0
```

```
Unnamed: 6: 0.0
```

```
Lab 7: 0.0
```

```
... (199 more lines)
```

Migrating the old ones

- Each old algorithm gets rewritten to the new interface, one at a time
- This is what real migration work looks like: unglamorous, and most of the job
- Keep the old copies until the new ones give the same answers
- Careful with generated rewrites: if you can't explain the diff, you can't ship it

→ cells 56, 57, 60, 61, 63, 64, 65

Migrating the old ones — cell 56

```
class calculator(alg):
    def __init__(self,name):
        alg.__init__(self,name)

    def apply(self,a_grade_book):
        raise NotImplementedError

class uncurved_letter_grade_percent(calculator):
    __grades_definition=[ (.97,"A+"),
                           (.93,"A"),
                           (.9,"A-"),
                           (.87,"B+"),
                           (.83,"B"),
                           (.8,"B-"),
                           ... (41 more lines)
```

Migrating the old ones — cell 57

```
a_grade_book.apply_calculator(uncurved_letter_grade_percent("Lab 2",max_grade=...  
# for k,a_student in a_grade_book.get_students().items():  
#     print (a_student.id_number(),a_student["Lab 2"],a_student["Lab 2 Letter"]..
```

Migrating the old ones — cell 60

```
class mean_std_calculator(calculator):  
    def __init__(self):  
        calculator.__init__(self, "Mean and Standard Deviation Calculator")  
  
    def apply(self, a_grade_book, grade_name, **kwargs):  
        grades = list()  
        for k, a_student in a_grade_book.get_students().items():  
            grades.append(a_student[grade_name].value())  
  
        a_grade_book[grade_name + " Mean"] = np.mean(grades)  
        a_grade_book[grade_name + " STD"] = math.sqrt(np.var(grades))
```

Migrating the old ones — cell 61

```
a_grade_book.apply_calculator(mean_std_calculator(),grade_name="Lab 3")
```

```
print (a_grade_book["Lab 3 Mean"])
```

```
print (a_grade_book["Lab 3 STD"])
```

```
46.4375
```

```
36.617906736322325
```

Migrating the old ones — cell 63

3

```
class mean_std_calculator(calculator):  
    def __init__(self, grade_name, cut_off=None):  
        self.__grade_name=grade_name  
        self.__cut_off=cut_off  
        calculator.__init__(self, "Mean and Standard Deviation Calculator")  
  
    def apply(self, a_grade_book, grade_name=None, cut_off=None, **kwargs):  
        if grade_name:  
            pass  
        else:  
            grade_name=self.__grade_name
```

... (18 more lines)

Migrating the old ones — cell 64

#1

```
class grade_summer(calculator):
    def __init__(self, prefix, n=None):
        self.__prefix=prefix
        self.__n=n
        calculator.__init__(self, "Sum Grades")

    def apply(self, a_gradebook, **kwargs):
        first=True

        for k, a_student in a_grade_book.get_students().items():
            if first:
                first=False

... (17 more lines)
```

Migrating the old ones — cell 65

#1

```
class curved_letter_grade(calculator):  
    __grades_definition=[ (.97,"A+"),  
                           (.93,"A"),  
                           (.9,"A-"),  
                           (.87,"B+"),  
                           (.83,"B"),  
                           (.8,"B-"),  
                           (.77,"C+"),  
                           (.73,"C"),  
                           (.7,"C-"),  
                           (.67,"C+"),  
                           (.63,"C"),  
                           ... (44 more lines)
```

Running them in sequence

- Stats → curve → summary, each reading what the last one left behind
- The order is now a *choice the user makes*, not something baked into the class
- Compose freely — the thing we couldn't do half an hour ago

→ cells 66, 67, 68, 73

Running them in sequence — cell 66

```
a_grade_book.apply_calculator(curved_letter_grade("Lab 3",  
                                                    a_grade_book["Lab 3 Mean"],  
                                                    a_grade_book["Lab 3 STD"]),  
                               overwrite=True)
```

Running them in sequence — cell 67

```
a_grade_book.print_students()
```

```
0 Student 0 Student Data
```

```
Lab 1: 0.0
```

```
Lab 2: 0.0
```

```
Lab 3: 0.0
```

```
Lab 4: 0.0
```

```
Exam 1: 0.0
```

```
Lab 5: 0.0
```

```
Unnamed: 6: 0.0
```

```
Lab 7: 0.0
```

```
... (215 more lines)
```

Running them in sequence — cell 73

```
a_grade_book.print_students()
```

```
0 Student 0 Student Data
```

```
Lab 1: 0.0
```

```
Lab 2: 0.0
```

```
Lab 3: 0.0
```

```
Lab 4: 0.0
```

```
Exam 1: 0.0
```

```
Lab 5: 0.0
```

```
Unnamed: 6: 0.0
```

```
Lab 7: 0.0
```

```
... (247 more lines)
```

Why `lambda` here?

- We hand the book a *way to make* the calculator, not the calculator itself
- Nothing is built until it's needed — lazy instantiation
- Same idea as Lecture 5's generators, and Lecture 10's lazy matrix product

New kinds are now cheap

- `forced_curve` — force a target distribution
- `boundry_grader` — take the boundaries as an argument instead of hard-coding them
- Neither one touches `grade_book`
- That's the test of a good interface: extension without modification

→ cells 75, 76, 78, 79, 81

New kinds are now cheap — cell 75

```
class mean_std_calculator(calculator):
    def __init__(self, grade_name, cut_off=None):
        self.__grade_name=grade_name
        self.__cut_off=cut_off
        calculator.__init__(self, "Mean and Standard Deviation Calculator")

    def apply(self, a_grade_book, grade_name=None, cut_off=None, **kwargs):
        if grade_name:
            pass
        else:
            grade_name=self.__grade_name

        if cut_off:
            pass
... (16 more lines)
```

New kinds are now cheap — cell 76

```
class grade_summer(calculator):
    def __init__(self, prefix, n=None):
        self.__prefix=prefix
        self.__n=n
        calculator.__init__(self, "Sum Grades")

    def apply(self, a_gradebook, **kwargs):
        first=True

        for k, a_student in a_grade_book.get_students().items():
            if first:
                first=False
                if self.__n:
                    labels=[self.__prefix+str(x) for x in range(1, self.__n)]
... (17 more lines)
```

New kinds are now cheap — cell 78

```
class forced_curve(calculator):
    def __init__(self, grade_name, cut_off=None):
        self.__grade_name=grade_name
        self.__cut_off=cut_off
        calculator.__init__(self, "Forced Curve Boundry Statistical Calculator"...

    def apply(self, a_grade_book, grade_name=None, cut_off=None, **kwargs):
        if grade_name:
            pass
        else:
            grade_name=self.__grade_name

        if cut_off:
            pass

... (30 more lines)
```

```
a_grade_book.apply_calculator(forced_curve(grade_name="Exam 1 Fixed"))
```

New kinds are now cheap — cell 81

```
class calculator(alg):
    def __init__(self,name):
        alg.__init__(self,name)

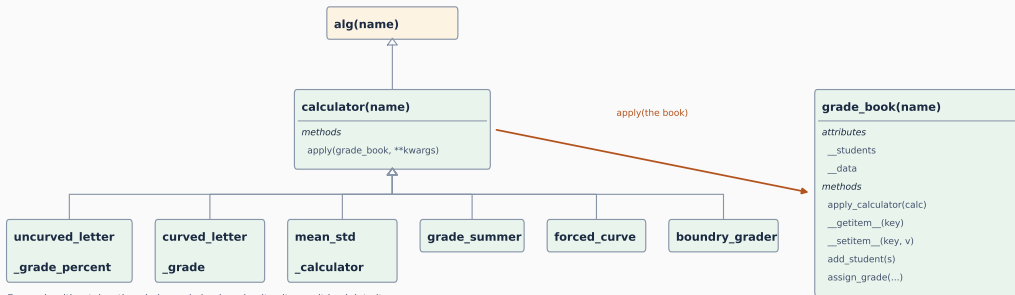
    def apply(self,a_grade_book):
        raise NotImplementedError

class boundry_grader(calculator):
    __max_grade=100.
    __grade_name=str()

    def __init__(self,grade_name,grades_definition,max_grade=100.):
        self.__max_grade=max_grade
        self.__grade_name=grade_name
... (29 more lines)
```

Where we ended up

One interface, and a blackboard to write on



Every algorithm takes the whole grade book and writes its result back into it, so the next one can pick it up. Adding a kind of calculation adds a class, nothing else.

Looking back

What this chapter was really about

- We wrote the gradebook three times, and each version taught the next
- Design iteration is **normal**. First versions are supposed to be wrong
- What you protect is the interface; implementations are cheap to replace
- Draw it first: Lab 6 asks for the UML before the blackjack code
- Next: enough software design — we turn to looking at data